

Chunker Memory: Why Media Pays and Text Doesn't

cantina-tools / Geneva quickstart | 2026-06-09

The shared mechanism

A Geneva chunker actor materializes the **full expansion of its work item** — up to 1024 source rows and every output row they produce — in one in-memory Arrow table before it returns (`ChunkerExpandActor.expand_batch`). The execution path is identical for every workload. Whether that buffer is negligible or catastrophic comes down to a **single term: the number of bytes carried on each output row**. That term is roughly the same for any text chunker and roughly a thousand times larger for a video or large-image chunker, which is the whole story.

The governing relationship

$$\text{Peak per-actor RAM} \approx B \times F \times S$$

- **B** = source rows per work item — fixed at **1024** in this Geneva build, identical for all workloads.
- **F** = output rows per source row (the chunker's fan-out) — comparable in range for text and media; if anything text fans out *more*.
- **S** = bytes per output row (the payload) — the one term that swings by ~1000× between text and media.

Because B is constant and F is similar, the memory difference is driven almost entirely by S.

Where text and media diverge

Dimension	Text chunking	Video / large-image chunking
Payload per output row (S)	A passage or token span: ~0.2–2 KB of UTF-8 plus light metadata.	<code>clip_bytes</code> (a 1-sec mp4, ~0.1–1 MB) <i>plus</i> a frame image (full-res PNG, 1–3 MB) → ~1–3 MB .
Arrow representation	Compact strings; in-memory size ≈ stored size; compresses well.	<code>large_binary</code> blobs; no cross-row dedup, poor compression; in-memory ≈ raw bytes.
Transient / decode memory	None — chunking is string slicing. Nothing is decoded or copied.	Per clip: the whole source video copied into a buffer (×2–3), a raw decoded frame (1080p RGB ≈ 6 MB), and off-heap ffmpeg/PyAV buffers the GC never sees.
Released incrementally?	Moot — the buffer is tiny to begin with.	No. <code>expand_batch</code> returns one blob, so nothing frees until the entire 1024-row batch finishes; peak = sum over the batch.

The same batch, two workloads

Take one work item (B = 1024 source rows) through both cases:

- **Text** — 1024 documents × 50 chunks each × 1 KB ≈ **~50 MB** per actor. Comfortable, even with a high fan-out.
- **Video** — 1024 videos × 5 clips each × 2 MB ≈ **~10 GB** per actor; across 10 actors, **~100 GB** of fleet RAM — the observed pile-up.

Note the text case fans out *10× more rows* yet uses ~200× *less* memory, purely because each row is ~2000× smaller. Fan-out is not the problem; payload is.

Why the design is sound for text

“Expand the whole batch, then write” implicitly assumes output rows are small and uniform — true for the canonical UDTF workloads it was built for: sentence/window splitting, tokenization, field extraction, dedup. There, a high row count is cheap because each row is a short string, and one IPC round-trip per work item is a worthwhile simplification. The pattern only turns into a liability when a row's payload is a media blob — the video/image case — where S is large enough to push B × F × S from megabytes into gigabytes.

Recommended solution

Because B is fixed and F is dictated by the chunking semantics, the durable answer is to stop the large bytes from ever sitting in the per-actor buffer. Three tiers, in increasing order of robustness:

- **1. Shrink the payload (applied).** Store the frame as a 512 px JPEG instead of a full-res PNG — the S term drops ~10–30× with no dependency changes. This keeps current runs healthy but only *reduces* the exposure; $B \times F \times S$ still scales with media size.
- **2. Move the bytes out of the row (recommended next step, within our control).** Have the chunker write each `clip_bytes` / frame to blob storage and emit only a lightweight **reference** (row-id or URI) — exactly the reference-only pattern the `videos` table already uses with `take_blobs`. The output row collapses to pointer-sized, so S returns to text scale and the buffer stays in megabytes regardless of media size; downstream stages fetch the bytes on demand on the worker. This *removes* the exposure rather than reducing it, with no Geneva change.
- **3. Bound the batch upstream (the proper Geneva fix).** Raise either with the Geneva team: (a) make `ChunkerExpandActor` **flush expanded rows in `max_rows_per_fragment`-sized increments** instead of returning the whole work item as one IPC blob — so the knob that already exists actually bounds actor RAM; or (b) expose the hard-coded `row_id_batch_size` (1024) as a refresh parameter so media UDTFs can shrink B and gain parallelism. Either caps memory by construction for *all* media workloads, not just this one.

Tier 1 is done; tier 2 is the recommended next change in this repo; tier 3 is the lasting fix and is worth filing upstream.